

# Web Application Vulnerability Assessment & Penetration Testing

OWASP Juice Shop — Professional Security Assessment Report

|                          |                                 |
|--------------------------|---------------------------------|
| <b>Prepared by</b>       | Sonu Shukla                     |
| <b>Assessment type</b>   | Controlled Web Application VAPT |
| <b>Target</b>            | OWASP Juice Shop (local lab)    |
| <b>Primary tools</b>     | Burp Suite, Docker              |
| <b>Assessment status</b> | Manual testing completed        |

This report documents security testing performed only against the local OWASP Juice Shop training application. Findings are based on the evidence captured during the assessment.

# 1. Executive Summary

A controlled manual VAPT exercise was performed against a locally hosted OWASP Juice Shop application. The assessment focused on understanding HTTP traffic, modifying requests in Burp Suite, validating application behavior, and documenting reproducible security findings.

Six findings were manually validated during the exercise: SQL injection authentication bypass, DOM-based XSS, IDOR/BOLA, sensitive data exposure through the FTP directory, weak authentication controls, and improper error handling with missing security headers.

The most significant issue observed was SQL injection in the login flow, which allowed authentication bypass in the lab environment. The remaining findings demonstrate weaknesses in authorization, client-side input handling, information exposure, authentication controls, and defensive HTTP configuration.

The purpose of this report is to present the technical evidence in a clear, interview-ready format and to show the testing methodology and remediation thinking used during the project.

## 2. Scope & Test Environment

| Item                 | Details                                                   |
|----------------------|-----------------------------------------------------------|
| Application          | OWASP Juice Shop                                          |
| Environment          | Local Docker container                                    |
| Target URL           | http://localhost:3000                                     |
| Testing approach     | Manual black-box web application testing                  |
| Proxy / testing tool | Burp Suite                                                |
| Container platform   | Docker                                                    |
| Scope limitation     | Testing was restricted to the local training application. |

## 3. Methodology

The assessment followed a practical workflow: establish the local test environment, browse the application through Burp Suite, identify interesting endpoints, replay and modify requests in Repeater, observe server responses, and capture evidence for confirmed issues.

| Phase               | Activity                                                       |
|---------------------|----------------------------------------------------------------|
| Reconnaissance      | Application browsing and HTTP history review.                  |
| Request analysis    | Inspection of parameters, endpoints, headers and responses.    |
| Manual validation   | Controlled modification of requests to test security controls. |
| Evidence collection | Screenshots of successful validation and relevant responses.   |
| Reporting           | Findings summarized with impact and remediation guidance.      |

## 4. Findings Overview

| ID   | Finding                                            | Severity | Affected Area          |
|------|----------------------------------------------------|----------|------------------------|
| F-01 | SQL Injection — Authentication Bypass              | Critical | POST /rest/user/login  |
| F-02 | DOM-Based Cross-Site Scripting                     | Medium   | /#/search?q=...        |
| F-03 | IDOR / BOLA — Broken Object-Level Authorization    | Medium   | GET /rest/basket/{id}  |
| F-04 | Sensitive Data Exposure                            | Medium   | /ftp / acquisitions.md |
| F-05 | Weak Authentication Controls                       | Medium   | /rest/user/login       |
| F-06 | Improper Error Handling + Missing Security Headers | Medium   | /rest/abc              |

Severity labels are practical assessment ratings for this training environment. They are intended to support prioritization rather than represent a formal CVSS calculation.

# F-01 — SQL Injection — Authentication Bypass

Severity: Critical    Affected area: POST /rest/user/login

## Description

The login endpoint accepted SQL syntax in the email parameter. A controlled authentication-bypass payload caused the application to return a successful authentication response associated with the administrator account in the local lab.

## Validation / Evidence

Burp Repeater was used to replay the login request and inspect the successful response. The response returned HTTP 200 and identified the administrator account. Authentication tokens were treated as sensitive and were redacted from report evidence.

## Impact

An attacker could potentially bypass normal authentication and gain access to an account without knowing its password. In a real application, this could lead to unauthorized access and compromise of protected functionality.

## Recommended Remediation

Use parameterized queries / prepared statements and never concatenate user-controlled input into SQL statements. Add negative security tests for authentication endpoints and monitor suspicious login activity.

## F-02 — DOM-Based Cross-Site Scripting

Severity: Medium    Affected area: Product search route: `/#/search?q=...`

### Description

A crafted search value was processed by client-side code and executed in the browser. The test produced an XSS alert in the local Juice Shop application, confirming client-side script execution.

### Validation / Evidence

The browser displayed the controlled XSS alert while the crafted search URL was visible. The evidence screenshot was captured during testing.

### Impact

Successful XSS can allow attacker-controlled JavaScript to execute in a victim's browser under the application's origin, potentially affecting user actions or exposing accessible browser-side data.

### Recommended Remediation

Use context-appropriate output encoding, avoid unsafe HTML sinks such as `innerHTML` where possible, validate input, and deploy a strong Content Security Policy as a defense-in-depth control.

## **F-03 — IDOR / BOLA — Broken Object-Level Authorization**

Severity: Medium    Affected area: GET /rest/basket/{id}

### **Description**

A basket request was replayed in Burp Repeater after changing the basket identifier. The server returned a different basket object instead of enforcing that the requested object belonged to the authenticated user.

### **Validation / Evidence**

The modified request to another basket identifier returned a successful response containing another basket object. The authorization token was not modified.

### **Impact**

A missing ownership check can allow an authenticated user to access another user's object by changing an identifier. This is a server-side authorization failure.

### **Recommended Remediation**

Perform an object-level authorization check on every request. The server should verify that the requested basket belongs to the authenticated user's identity before returning data.

## F-04 — Sensitive Data Exposure

Severity: Medium    Affected area: /ftp / acquisitions.md

### Description

The application exposed an FTP-style directory containing internal files. The acquisitions.md file was directly accessible and contained information marked as confidential.

### Validation / Evidence

The browser accessed /ftp and opened acquisitions.md without an additional authorization control. The evidence screenshot shows the publicly accessible confidential document.

### Impact

Public access to internal documents can disclose business information and provide useful context to an attacker for further attacks.

### Recommended Remediation

Remove sensitive files from the web-accessible directory, disable directory listing, enforce authorization for protected resources, and prevent backups/internal documents from being deployed to the public web root.

## **F-05 — Weak Authentication Controls — No Effective Lockout Observed**

Severity: Medium    Affected area: Login flow /rest/user/login

### **Description**

Repeated failed login attempts were performed against the lab login flow. No visible account lockout or CAPTCHA/rate-limit response was observed after the test sequence.

### **Validation / Evidence**

The final evidence screenshot shows the login interface after repeated failed attempts. The assessment did not observe a protective lockout/CAPTCHA response.

### **Impact**

Without effective rate limiting or adaptive controls, repeated password-guessing attempts can be performed more easily. The practical risk depends on password strength and other controls.

### **Recommended Remediation**

Apply server-side rate limiting, progressive delays, account protection controls, MFA for sensitive accounts, and monitoring/alerting for repeated authentication failures.

## **F-06 — Improper Error Handling + Missing Security Headers**

Severity: Low/Medium    Affected area: /rest/abc

### **Description**

An invalid API path generated a detailed 500 error page containing internal application and framework path information. Response headers also showed that Content-Security-Policy and Strict-Transport-Security were not present in the observed response, while X-Content-Type-Options and X-Frame-Options were present.

### **Validation / Evidence**

The /rest/abc response displayed a stack trace with internal paths. Response headers were reviewed in browser developer tools.

### **Impact**

Detailed error messages help attackers understand application internals. Missing defensive headers reduce browser-side security hardening.

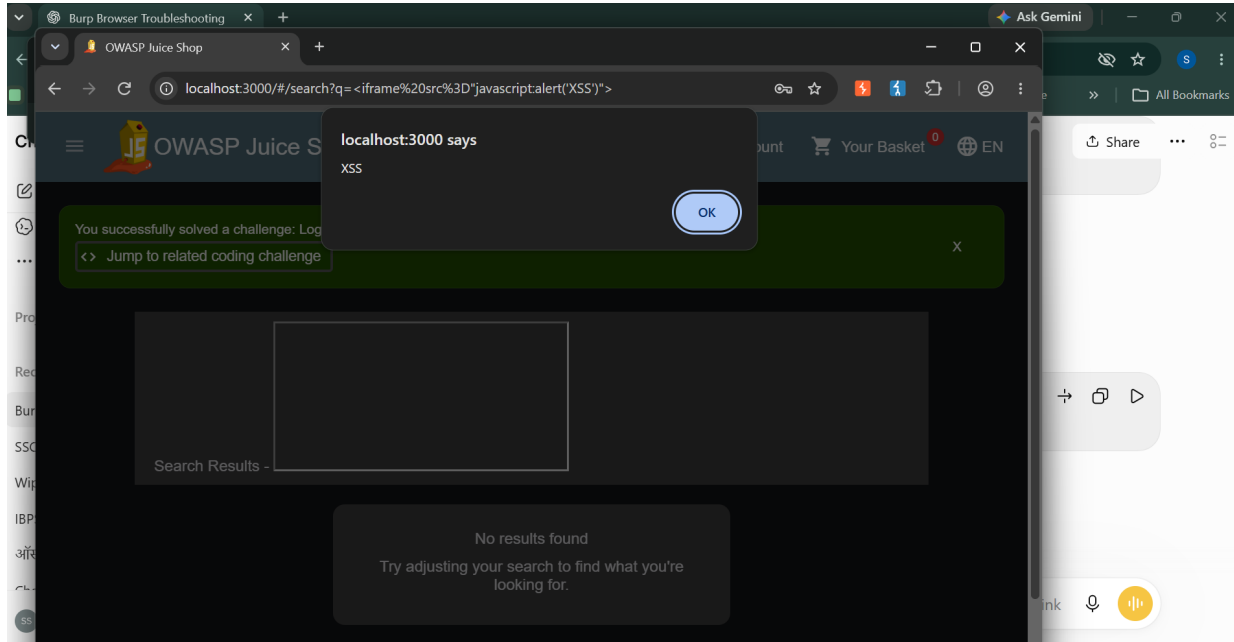
### **Recommended Remediation**

Return generic error messages to clients and log detailed exceptions server-side. Add an appropriate Content-Security-Policy and Strict-Transport-Security policy where HTTPS is used, and maintain a consistent security-header baseline.

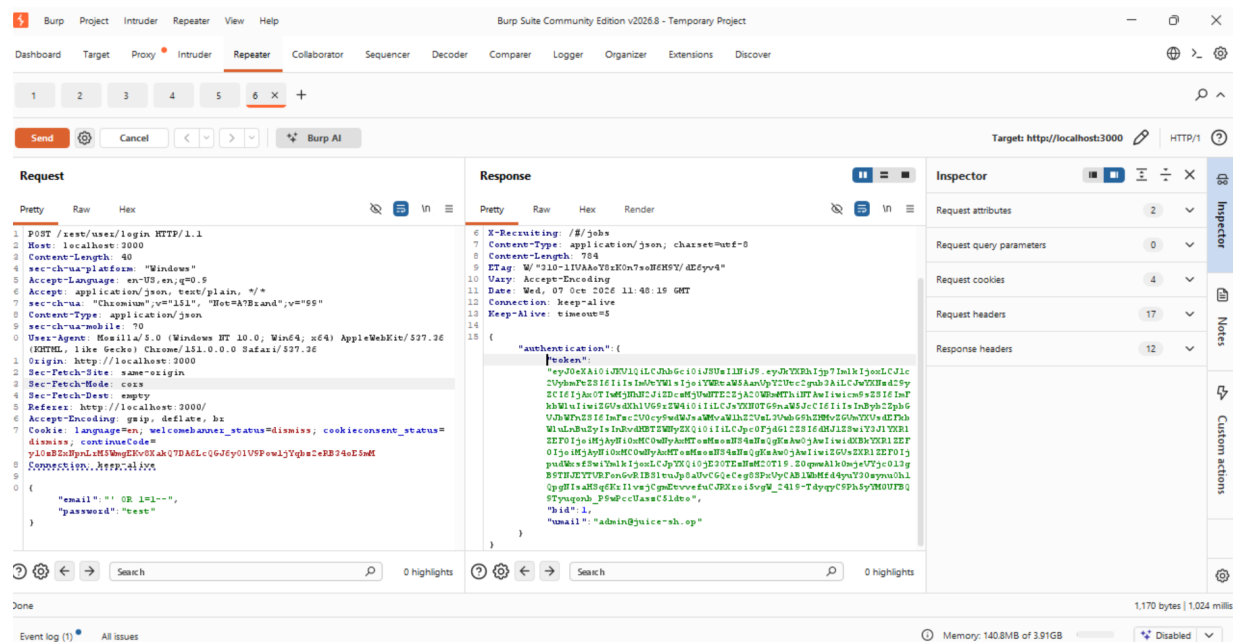
## 5. Evidence Appendix

The following screenshots were captured during the assessment. Sensitive authentication material was redacted before inclusion.

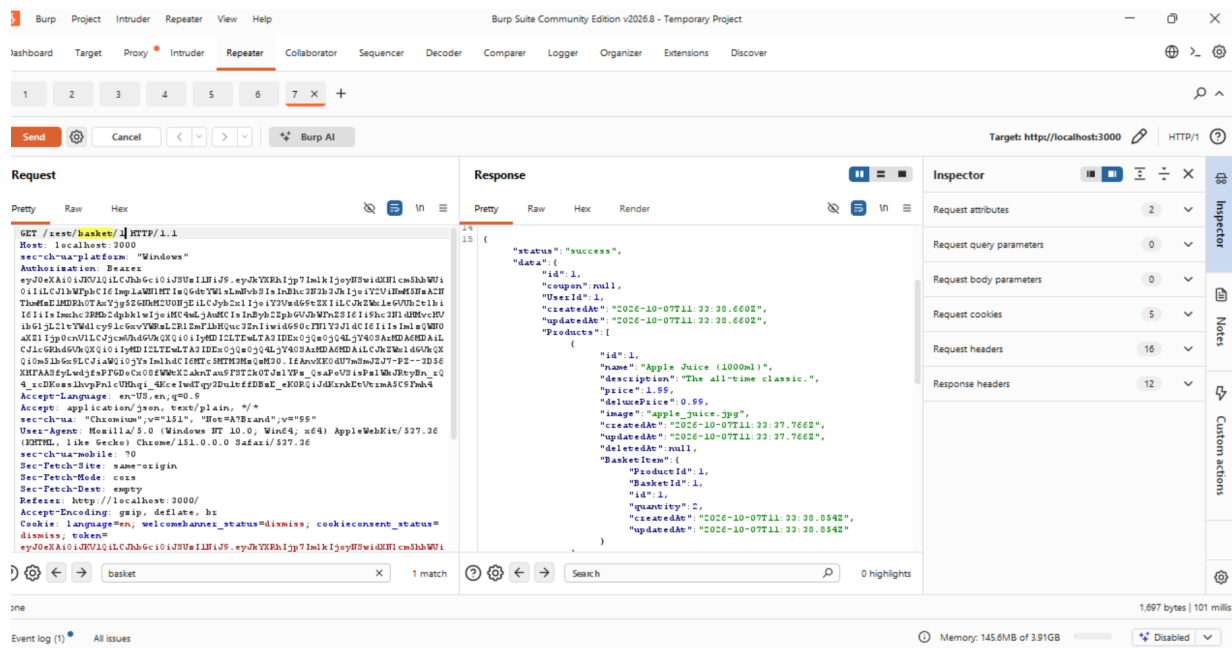
### Evidence A — DOM-Based XSS



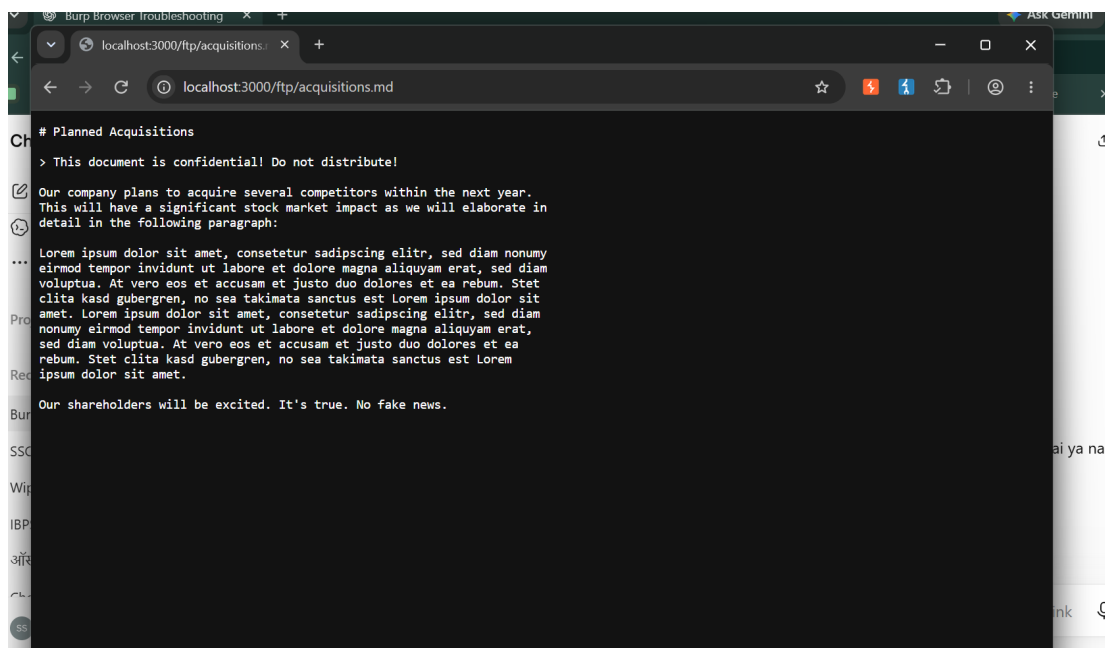
### Evidence B — SQL Injection Authentication Bypass



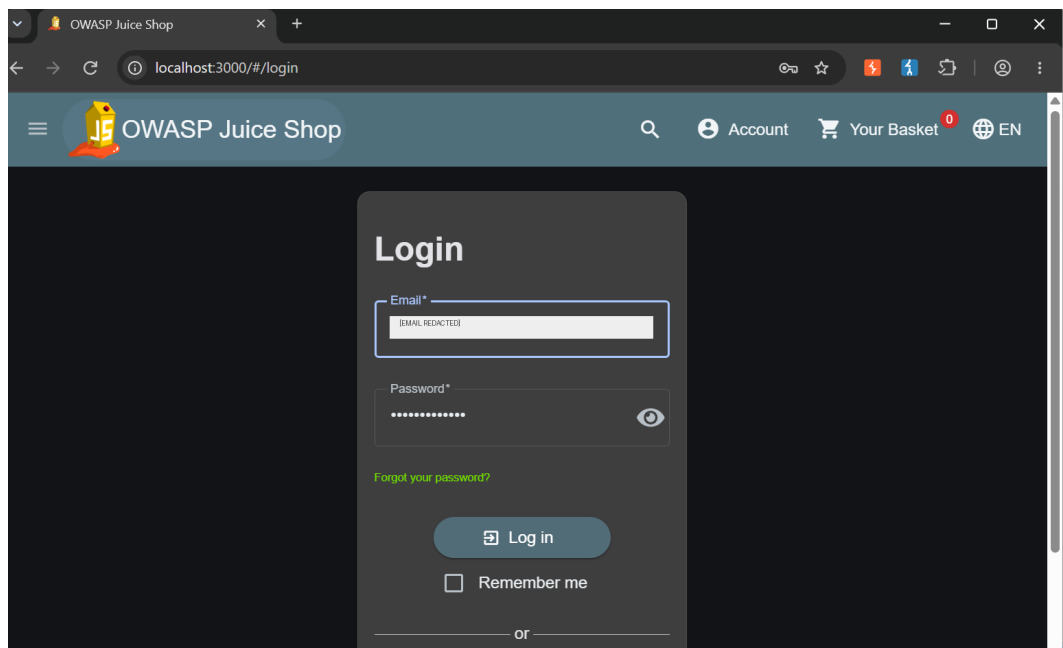
### Evidence C — IDOR / BOLA



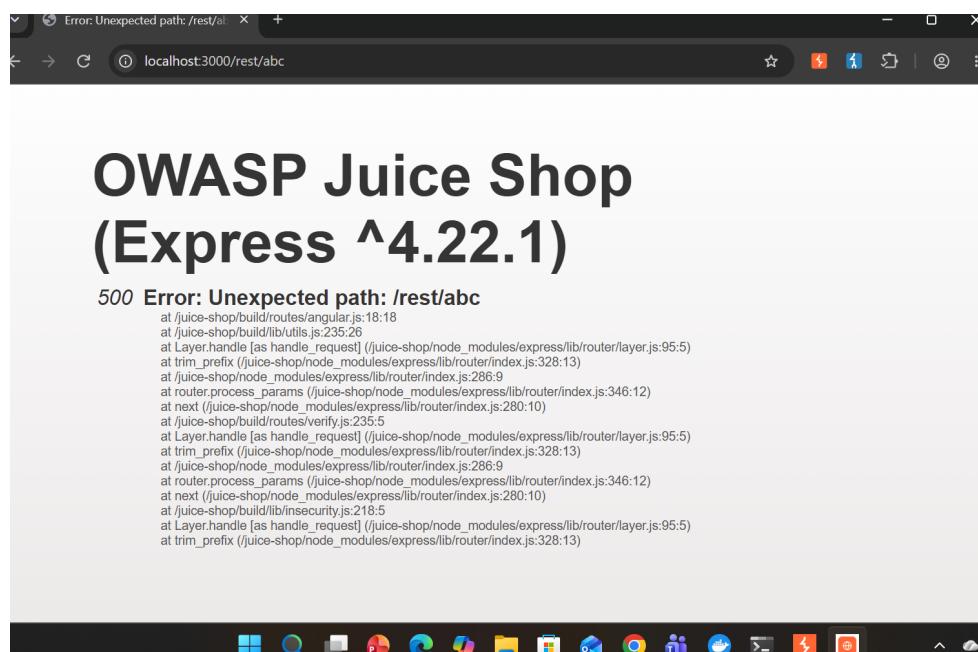
## Evidence D — Sensitive Data Exposure



## Evidence E — Weak Authentication Controls



## Evidence F — Improper Error Handling



## 6. Remediation Priorities

Priority 1: Fix the SQL injection in the authentication flow using parameterized queries.

Priority 2: Enforce server-side object ownership checks for basket and other object endpoints.

Priority 3: Remove publicly accessible confidential files and harden file/directory access.

Priority 4: Improve authentication protections with rate limiting and additional account safeguards.

Priority 5: Fix unsafe client-side HTML handling and strengthen browser security policy.

Priority 6: Replace detailed client-facing errors with generic responses and establish a security-header baseline.

## 7. Conclusion

The assessment demonstrated a complete practical VAPT workflow on a controlled web application: traffic inspection, request manipulation, manual validation, evidence capture, impact analysis, and remediation planning. The exercise produced six reproducible findings and provides a strong foundation for discussing web security testing in a placement interview.

The assessment was limited to the local OWASP Juice Shop training environment and should not be interpreted as an assessment of any external or production system.